

CachePilot：基于 Mooncake 的 KVCache 复用与调度评测系统

技术文档 / 实机测试记录 / 演示说明

2026 年 Mooncake 赛题参赛作品

仓库：<https://github.com/imperson123/CachePilot>

1 工作内容概览

本文记录 CachePilot 围绕 Mooncake Store、KVCache 复用和检索调度开展的评测系统实现与实机测试结果。文档样式参考用户提供的复现记录类报告，整体采用“概览-模块-结果-小结”的紧凑结构：先给出系统组成与实验环境，再给出真实 Store Benchmark、可控 Prefix Reuse workload 评估和 Retrieval Scheduler 策略评估，最后总结当前边界与后续扩展方向。

表 1: CachePilot 工作内容总览

方向	模块	完成内容
Mooncake Store 性能	Store Benchmark	包装官方 store_kv_bench.py，完成真实 Mooncake Store verify_write 与 read_perf 实机测试
KVCache 复用	Prefix Reuse Evaluation	构造 prefix length 与 cache hit ratio 可控 workload，分析 TTFT 改善趋势
检索调度	Retrieval Scheduler Evaluation	对比 Random、Nearest、Reuse-aware 与 CachePilot 策略的延迟、远程流量和热点比
工程展示	Streamlit Dashboard	读取 CSV 与图表，提供可录屏、可截图的演示界面

与只提交脚本不同，CachePilot 的目标是形成一个可复现的评测套件。仓库中保留了命令行脚本、CSV 结果、图表、Dashboard 与技术文档，便于赛事评审或社区维护者快速判断其接口边界、运行方式和当前结果可信度。

2 赛题背景与问题定位

Mooncake 赛题关注 KVCache 存储、复用、传输和调度能力。大模型服务进入长上下文、多轮对话与高并发部署后，KVCache 的存储和检索会影响 TTFT、吞吐、尾延迟与系统成本。赛题材料也明确鼓励参赛作品提供可复现性能数据、运行说明、实验报告和开源贡献材料，而不是停留在概念设计。

CachePilot 选择的切入点是 Mooncake Store 的性能、高可用与 HiCache 相关优化方向中的“benchmark 补充与调度评估”。该切入点相对稳妥：不侵入 Mooncake 核心 C++/RDMA 路径，而是复用官方 Store benchmark，先建立真实 I/O 基线，再扩展上层 Prefix Reuse 与 Retrieval Scheduler 策略评估。

表 2: 赛题关注点与 CachePilot 对应关系

赛题关注点	CachePilot 对应模块	说明
Store put/get 性能	mooncake_store_runner.py	调用官方 store_kv_bench.py, 记录 req/s、MiB/s、P50/P95/P99 等指标
可复现实验	scripts/run_*.sh	保留一键运行脚本、原始日志和结构化 CSV
Prefix 复用收益	prefix_reuse_benchmark.py	可控分析 prefix length、hit ratio 与 TTFT 关系
检索与副本调度	retrieval_scheduler_sim.py	比较不同调度策略的延迟、远程流量和热点负载
演示材料	dashboard.py	提供可视化面板, 用于录屏和文档截图

3 系统设计

CachePilot 遵循非侵入式设计: 它不修改 Mooncake Store、Transfer Engine 或 RDMA 传输核心, 而是在外层建立评测流水线。真实 Store 评测部分直接调用 Mooncake 官方脚本; 策略评估部分使用 CSV 和参数化 workload 作为输入, 最后统一输出到 results/csv、results/logs 和 results/figures。

3.1 总体架构

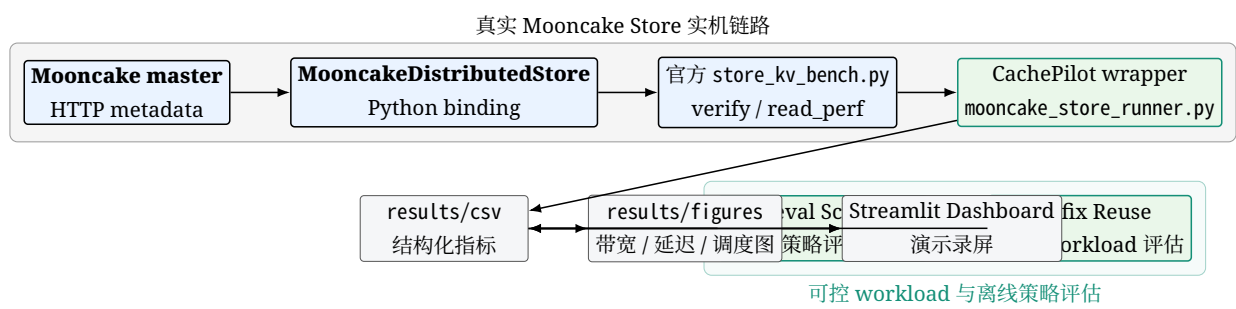


图 1: CachePilot 总体架构: 上层复用 Mooncake 官方 Store benchmark 建立真实 I/O 基线, 下层扩展 Prefix Reuse 与 Scheduler 评估。

3.2 真实 Store 评测链路

Store Benchmark 是本项目的实机实测核心。运行时首先启动 mooncake_master 并启用 HTTP metadata server; 随后 CachePilot wrapper 调用官方 store_kv_bench.py, 将 stdout/stderr 保存到日志文件, 再用解析器提取 req/s、MiB/s、延迟分位、misses 与 verify failures。

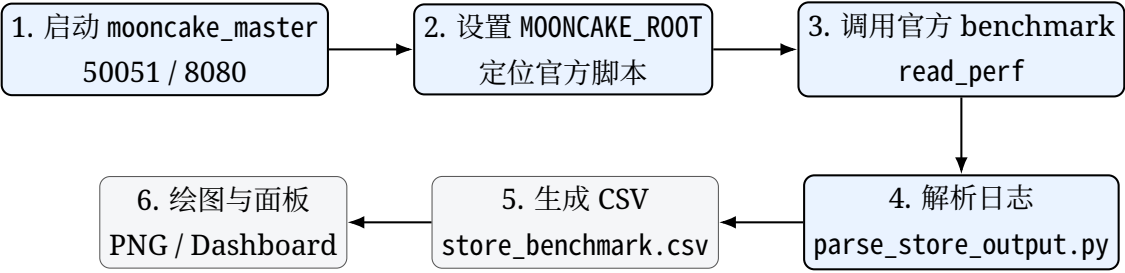


图 2: 真实 Store Benchmark 数据采集流程。

3.3 代码目录与功能划分

表 3: CachePilot 关键文件说明

文件 / 目录	功能
benchmark/mooncake_store_runner.py	通过 subprocess 调用 Mooncake 官方 store_kv_bench.py, 完成参数矩阵运行、日志保存和 CSV 汇总
benchmark/parse_store_output.py	从官方 benchmark 输出中解析 req/s、kv/s、MiB/s、latency 分位、misses 和 verify failures
benchmark/prefix_reuse_benchmark.py	构造 prefix length 与 hit ratio 参数矩阵, 评估 prefix 复用对 TTFT 的影响趋势
benchmark/retrieval_scheduler_bench.py	生成多节点 KV block workload, 对比 Random、Nearest、Reuse-aware 与 CachePilot 策略
dashboard.py	读取 CSV 和图表, 展示 Store 实测、Prefix Reuse、Scheduler 和 Implementation 页面
scripts/run_all.sh	一键执行 Store、Prefix、Scheduler、Plot 全流程

4 实验环境与运行配置

本次实测在 AutoDL 容器实例上完成，使用单卡 RTX 4090 机器。当前结果的定位是“单机 TCP Mooncake Store benchmark”，不等同于 RDMA、多机或完整 LLM serving 端到端结果。

表 4: 真实 Store Benchmark 实验环境

配置项	实验设置
平台	AutoDL 容器实例
GPU	RTX 4090 24GB
CPU / 内存	16 核 / 120GB
系统	Ubuntu 22.04
Python / CUDA	Python 3.10 / CUDA 11.8
Mooncake 包	mooncake-transfer-engine-non-cuda
协议	TCP
Master	HOST_IP:50051
Metadata	http://HOST_IP:8080/metadata
测试脚本	mooncake-store/benchmarks/store_kv_bench.py
CachePilot 仓库	https://github.com/imperson123/CachePilot

4.1 运行命令摘要

Listing 1: Mooncake master 与 CachePilot 运行命令摘要

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libstdc++.so.6
export MOONCAKE_ROOT=/root/autodl-tmp/mooncake_real_run/Mooncake
export HOST_IP=$(hostname -I | awk '{print $1}')
```

```
mooncake_master \
  --port=50051 \
  --enable_http_metadata_server=true \
  --http_metadata_server_host=0.0.0.0 \
  --http_metadata_server_port=8080 \
  --eviction_high_watermark_ratio=0.95

python benchmark/mooncake_store_runner.py \
  --mooncake-root $MOONCAKE_ROOT \
  --scenario read_perf \
  --io-api plain \
  --local-hostname ${HOST_IP}:50071 \
  --metadata-server http://${HOST_IP}:8080/metadata \
  --master-server ${HOST_IP}:50051 \
  --global-segment-size 536870912 \
  --local-buffer-size 268435456 \
  --value-sizes 4096,65536,1048576,4194304 \
  --batch-sizes 1,4,8,16 \
  --runtime 5 \
  --nr-objects 64
```

5 真实 Mooncake Store Benchmark 结果

5.1 功能验证结果

首先执行 `verify_write` 场景，写入后读回并验证 `payload` 正确性。该步骤用于确认 Python binding、master、metadata server 和 Store client 链路全部可用。

表 5: 真实 `verify_write` 结果

value size	batch	req/s	kv/s	MiB/s	p50/ms	p95/ms	p99/ms	校验
4KB	4	2482.98	9931.94	38.80	0.260	0.718	0.781	misses=0, failures=0

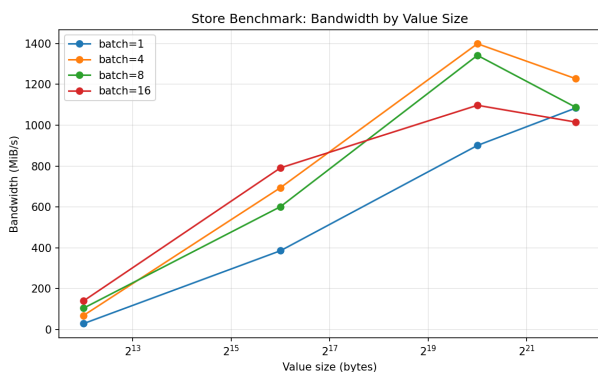
该结果说明 CachePilot 并非只生成图表，而是真实驱动 Mooncake Store 完成 put/get 验证。后续 `read_perf` 结果均基于同一套 master 与 metadata 链路。

5.2 read_perf 结果总览

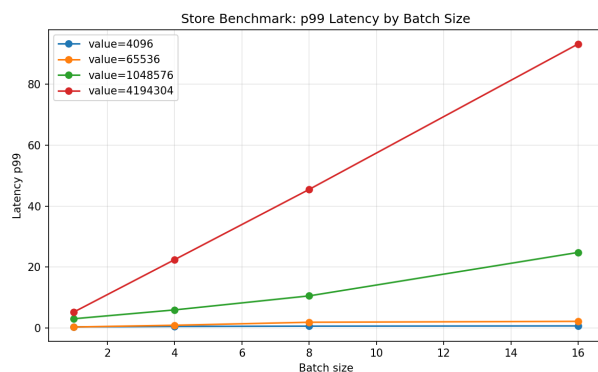
表 6: 真实 `read_perf` 代表性结果

value size	batch	MiB/s	p50/ms	p95/ms	p99/ms
4KB	1	28.78	0.106	0.225	0.380
4KB	16	139.86	0.352	0.661	0.703
64KB	4	694.43	0.304	0.503	0.917
1MB	4	1399.08	2.574	3.785	5.956
4MB	4	1227.74	12.251	15.440	22.456
4MB	16	1015.36	52.517	88.238	93.267

16 组 `read_perf` 全部 `return_code=0`，且 `misses=0`、`verify_failures=0`。结果体现出清晰趋势：小对象受请求开销影响明显；对象增大后带宽迅速上升；1MB、batch=4 时达到峰值 1399.08 MiB/s；继续增大至 4MB 或提高 batch 后，吞吐仍较高，但 P99 尾延迟明显上升。



(a) 带宽随 value size 变化



(b) P99 延迟随 batch size 变化

图 3: 真实 Mooncake Store `read_perf` 结果图。

5.3 完整结果矩阵

表 7: 真实 read_perf 16 组完整结果

value size	batch	req/s	kv/s	MiB/s	p50/ms	p99/ms
4KB	1	7368.67	7368.67	28.78	0.106	0.380
4KB	4	4392.48	17569.92	68.63	0.172	0.539
4KB	8	3339.45	26715.56	104.36	0.214	0.626
4KB	16	2237.75	35803.93	139.86	0.352	0.703
64KB	1	6168.78	6168.78	385.55	0.138	0.354
64KB	4	2777.72	11110.90	694.43	0.304	0.917
64KB	8	1202.69	9621.50	601.34	0.641	1.901
64KB	16	791.38	12662.14	791.38	0.917	2.199
1MB	1	901.03	901.03	901.03	1.019	3.046
1MB	4	349.77	1399.08	1399.08	2.574	5.956
1MB	8	167.78	1342.24	1342.24	5.224	10.559
1MB	16	68.60	1097.65	1097.65	11.010	24.824
4MB	1	271.12	271.12	1084.48	3.623	5.242
4MB	4	76.73	306.94	1227.74	12.251	22.456
4MB	8	33.99	271.89	1087.55	26.942	45.513
4MB	16	15.86	253.84	1015.36	52.517	93.267

6 Prefix Reuse 与 Scheduler 评估

Store Benchmark 给出了真实 I/O 基线；在此基础上，CachePilot 进一步提供两个策略评估模块，用于服务赛题对 KVCache 复用和调度的关注。需要说明的是，这两个模块不是完整分布式 LLM serving 端到端实测，而是基于可控 workload 的离线策略评估，用于说明评测框架如何扩展到 prefix hit ratio、TTFT、远程流量和热点等指标。

6.1 Prefix Reuse Evaluation

Prefix Reuse 评估构造多个请求共享相同 system prompt 或长上下文 prefix 的场景。无缓存时，每个请求都需要重新 prefill；命中 Prefix Cache 后，请求可通过 Store 取回 prefix KV，从而减少 TTFT。模块根据 prefix tokens、cache hit ratio、KV bytes per token 和 Store get latency 参数输出 TTFT 改善趋势。

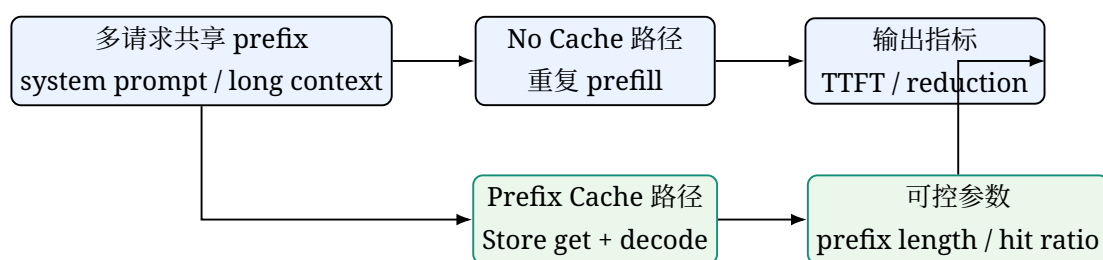


图 4: Prefix Reuse Evaluation 的可控 workload 设计。

6.2 Retrieval Scheduler Evaluation

Retrieval Scheduler 模块构造多节点 KV block metadata, 包括 reuse_count、importance_score、replica_list、estimated_fetch_latency_ms 和 size_mb。CachePilot 策略使用综合分数选择更合适的副本或源节点：

$$\text{score} = \alpha \cdot \text{importance} + \beta \cdot \text{reuse} - \gamma \cdot \text{fetch latency}. \quad (1)$$

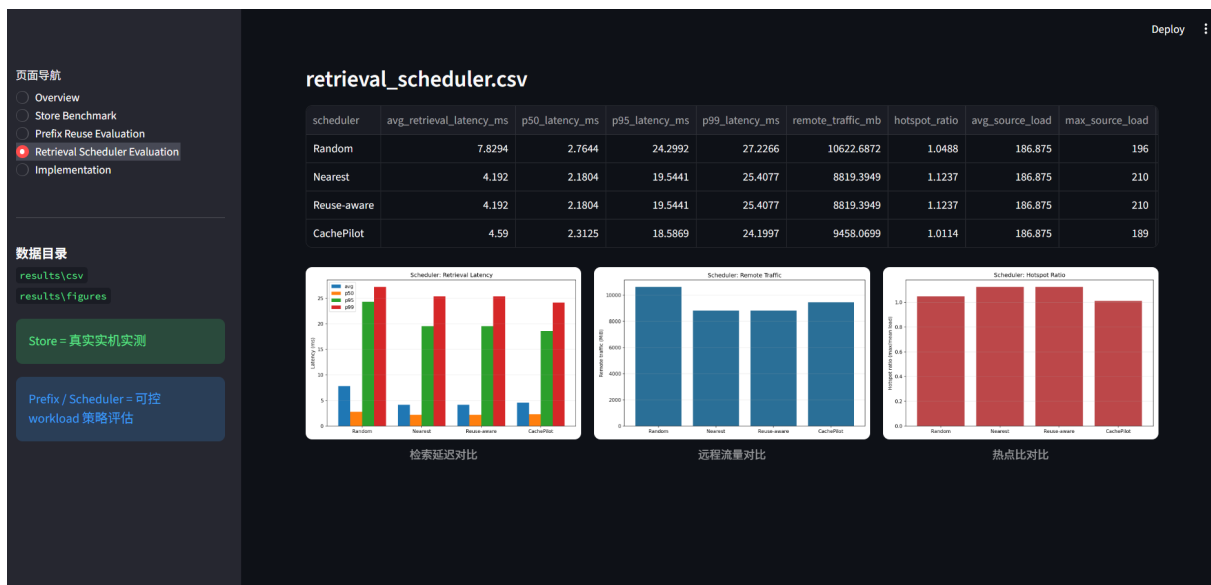


图 5: Dashboard 中的 Retrieval Scheduler Evaluation 页面。该页面用于对比不同策略的检索延迟、远程流量和热点比。

7 可视化 Dashboard 与演示

为了满足比赛演示视频和文档截图需求，CachePilot 增加了 Streamlit Dashboard。Dashboard 读取 results/csv 与 results/figures，以页面导航方式展示 Overview、Store Benchmark、Prefix Reuse Evaluation、Retrieval Scheduler Evaluation 和 Implementation。相较直接展示命令行日志，Dashboard 能更清楚地区分“真实 Store 实机实测”和“可控 workload 策略评估”。



图 6: Dashboard 中的 Store Benchmark 页面，展示真实 Store 带宽和 P99 延迟趋势。

7.1 演示视频建议流程

1. 打开 GitHub 仓库，说明 CachePilot 是非侵入式 benchmark/evaluation suite;
2. 展示 results/csv/store_verify_real.csv 和 store_benchmark.csv, 强调 return_code=0, misses=0, verify failures=0;
3. 打开 Dashboard 的 Store Benchmark 页面，讲解 1MB batch=4 峰值带宽和 4MB batch=16 尾延迟上升;
4. 切换到 Prefix Reuse 与 Scheduler 页面，说明它们是可控 workload 策略评估;
5. 最后展示 scripts/run_all.sh 与 README.md，强调可复现与可继续扩展。

8 问题处理与工程记录

真实 Store 运行过程中遇到两个工程兼容问题。第一，AutoDL 镜像中的 libstdc++ 与 pip 安装的 mooncake 包要求不匹配, 缺少 GLIBCXX_3.4.30; 通过 LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libstdc++.so.6 解决。第二，仓库中的 store_kv_bench.py 与 pip 包版本存在接口差异，如 get_alloc_func_addr 和 nof_replica_num 字段兼容问题；在本地实测中对脚本做了兼容补丁。

表 8: 实机运行中的问题与处理

问题	表现	处理方式
GLIBCXX_3.4.30 缺失	Python import Mooncake binding 报错	使用系统新版 libstdc++.so.6 进行 LD_PRELOAD
Python binding 与仓库脚本接口差异	get_alloc_func_addr 或 nof_replica_num 报错	对官方 benchmark 本地打兼容补丁，仅用于实测环境
4MB 高 batch 初次失败	默认 segment / buffer 偏小	将 global-segment-size 调至 512MB，local-buffer-size 调至 256MB

这些问题本身也说明了补充 benchmark 文档和自动化脚本的价值：评审或社区维护者需要知道依赖版本、运行参数和失败边界，才能复现实验结果。

9 结果分析与结论

本次 CachePilot 工作完成了从仓库实现、实机验证、结果可视化到演示材料的一条完整链路。Store Benchmark 的结果最关键：verify_write 成功，read_perf 16 组全部 return_code=0，并且 misses 与 verify failures 均为 0。由此可以确认该项目不是单纯生成模拟图，而是真实调用了 Mooncake Store 与官方 benchmark。

表 9: 当前结论与后续扩展

结论项	内容
真实性	Store Benchmark 已完成单机 TCP 实机实测，结果来自 Mooncake 官方 store_kv_bench.py
性能趋势	1MB、batch=4 取得最高带宽 1399.08 MiB/s；4MB 高 batch 下尾延迟显著上升
工程价值	提供一键脚本、CSV、日志、图表和 Dashboard，便于复现和评审
当前边界	尚未覆盖 RDMA、多机、zcopy、write_perf 和 mixed_rw 全矩阵
后续计划	扩展多协议、多节点与 SGLang HiCache 场景；用真实 Store 延迟进一步标定 Prefix Reuse 与 Scheduler 模型

总体而言，CachePilot 当前版本适合作为 Mooncake Store 性能评测与 KVCache 策略评估的基础贡献。它保留了真实 benchmark 数据，也提供了面向比赛演示的视频素材和技术文档入口；后续可在 PR 中以 benchmarks/cachepilot/ 的形式作为非侵入式 benchmark suite 进入社区讨论。